

ARGOMENTO 12

Piani di esecuzione di query

Il problema (intuizione)

Quando scriviamo una query SQL, come ad esempio:

```
SELECT nome, cognome  
FROM studenti  
WHERE corso = 'Informatica'  
ORDER BY cognome;
```

il DBMS non «esegue» direttamente la query, dato che essa è per natura dichiarativa (ci dice **cosa** vogliamo ottenere, non **come**)

Il suo compito è di **costruire un piano** per decidere **come** ottenere i dati richiesti nel modo più efficiente possibile

Il ruolo dell'ottimizzatore del DBMS

- Il *query optimizer* del DBMS ha un ruolo chiave nel decidere come eseguire una query SQL nel modo più efficiente possibile, valutando diverse possibili strategie di esecuzione.
- In particolare:
 - Genera piani di esecuzione alternativi, identificando le operazioni (WHERE, JOIN, ORDER BY, ...) e variando l'ordine della loro esecuzione
 - Calcola i costi di ognuno di questi piani (tipicamente in termini di operazioni di I/O)
 - Seleziona il piano ottimale

Il ruolo dell'ottimizzatore del DBMS

- Esempi di scelte che debbono essere fatte:
 - Come accedere alle tabelle: *full table scan* o *indice*?
 - Quando applicare le condizioni filtro (WHERE): per es. primo o dopo un determinato join?
 - Come eseguire i join: nested loop join, hash join, sort merge join, ...
 - Come ordinare i dati: usare un indice già ordinato o eseguire un sort esplicito?

Esecuzione di query SQL

- Per *stimare* il costo dell'esecuzione di un piano, viene calcolato il **costo** di ogni singolo passo in termini di I/O
- Per farlo, l'*optimizer* utilizza innanzitutto le statistiche che il DBMS aggiorna regolarmente:
 - cardinalità delle tabelle ($|R|$)
 - cardinalità dell'indice (valori distinti per ogni colonna indicizzata)
 - distribuzione dei valori in ogni colonna (quanti valori distinti, *min*, *max*, ...)
 - *selectivity factor*: stima della percentuale di tuple che supereranno una clausola di filtro WHERE per un certo valore

Cosa dobbiamo sapere per il calcolo

- Inoltre, l'*optimizer* deve conoscere anche:
 - Quali indici sono disponibili (e di che tipo)
 - Il numero di pagine P_R utilizzate per memorizzare ogni relazione (ottenuto da P / t_R)

Albero di esecuzione della query

- Per rappresentare i passi di esecuzione di una query, spesso si usa una rappresentazione ad albero:
 - Le foglie sono le relazioni di partenza (con la modalità di accesso, es. full scan o indice)
 - I nodi intermedi corrispondono a operazioni algebriche sulla/e relazioni coinvolte (join, filtri, aggregazioni, ...)
 - A ogni nodo intermedio viene associato un costo (con il numero di righe della relazione intermedia)
 - La radice corrisponde all'output della query (che a sua volta è una relazione)

Esempio su DB Sailors

Sailors (sid, sname, srating, age)

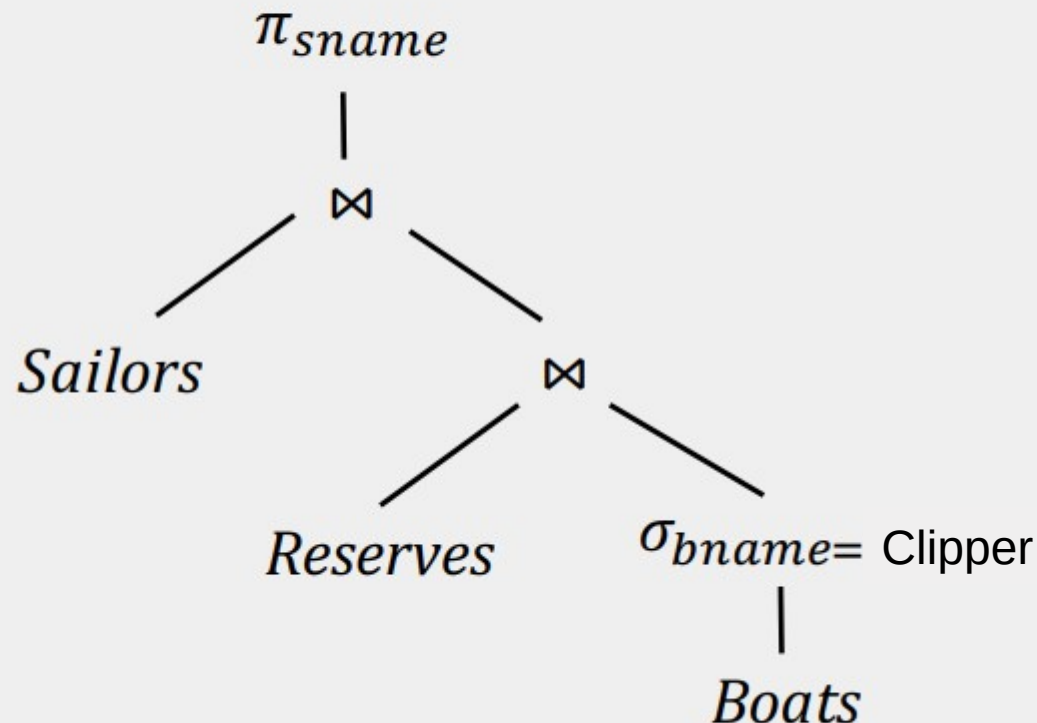
Boats(bid, bname, color)

Reserves(sid, bid, date, rname)

```
select s.sname
from sailors s
join reserves r on r.sid = s.sid
join boats b on b.bid = r.bid
where b.bname = 'Clipper';
```


Piano logico

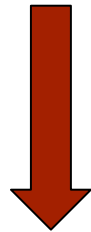
Q1: *What are the names of the sailors who have reserved boat with name Clipper?*



Piano di esecuzione

Per conoscere il piano di esecuzione scelto dal DBMS, si può usare il comando

EXPLAIN ANALYZE format=TREE



```
-> Nested loop inner join (cost=2.03 rows=2.5) (actual time=0.126..0.139 rows=3 loops=1)
  -> Nested loop inner join (cost=1.15 rows=2.5) (actual time=0.111..0.12 rows=3 loops=1)
    -> Filter: (b.bname = 'Clipper') (cost=0.65 rows=1) (actual time=0.0781..0.0799 rows=1 loops=1)
      -> Table scan on b (cost=0.65 rows=4) (actual time=0.0689..0.073 rows=4 loops=1)
    -> Covering index lookup on r using FK2_R (bid=b.bid) (cost=0.501 rows=2.5) (actual time=0.0304..0.0366 rows=3 loops=1)
  -> Single-row index lookup on s using PRIMARY (sid=r.sid) (cost=0.29 rows=1) (actual time=0.0055..0.00553 rows=1 loops=3)
```

Step 1: scan della tabella e equality search

Table scan **on** b (cost=0.65 rows=4)
(actual **time**= 0.0689..0.073 rows=4 loops=1)

Filter: (b.bname = 'Clipper') (cost=0.65 rows=1)
(actual **time**=0.0781..0.0799 rows=1 loops=1)

Il database ha letto tutte e 4 le tuple di boats e ne ha selezionata 1 (la barca che si chiama Clipper).

Potenziiale miglioramento: la presenza di un indice su *b.bname* lo avrebbe convertito in un più veloce **Equality Search** su indice.

Il filtro produce una sola riga di output.

Step 2: join tra Boat e Reserves

Nested **loop inner join** (cost=1.15 rows=2.5) (actual time=0.111..0.12 rows=3 loops=1)

Covering **index** lookup on r using FK2_R (bid=b.bid) (cost=0.501 rows=2.5) (actual time=0.0304..0.0366 rows=3 loops=1)

Il JOIN tra la barca 'Clipper' e le tuple di Reserves ha prodotto **3** prenotazioni per la barca Clipper.

L'uso di un **covering index** (ovvero un indice che fa match con gli attributi del filtro) su Reserves velocizza notevolmente il JOIN.

Step 3: JOIN con Sailors e output

Nested **loop inner join** (cost=**2.03** **rows**=**2.5**) (actual **time**=0.126..0.139 **rows**=3 loops=1)

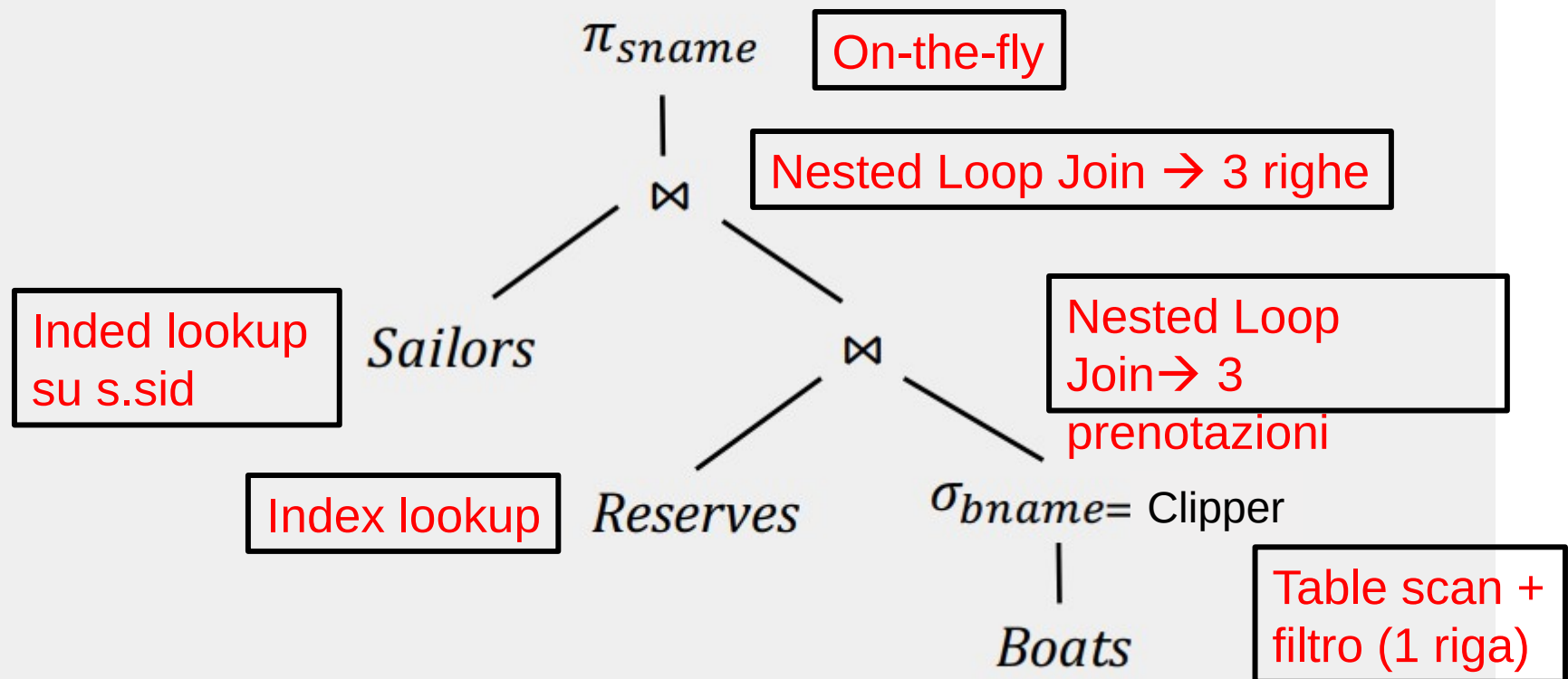
Single-**row index** lookup **on** s **using PRIMARY** (sid=r.sid) (cost=**0.29** **rows**=1) (actual **time**=0.0055..0.00553 **rows**=1 loops=3)

Il lookup su **S** è stato eseguito **3 volte** (una volta per ogni prenotazione, per la quale è stato trovato il marinaio corrispondente), e come risultato ha recuperato con successo i 3 marinai.

	A-Z sname ▼
1	Dustin
2	Iubber
3	Horatio

Piano di esecuzione

Q1: *What are the names of the sailors who have reserved boat with name Clipper?*



Calcolo costi di I/O

Sailors (sid, sname, srating, age)

Boats(bid, bname, color)

Reserves(sid, bid, date, rname)

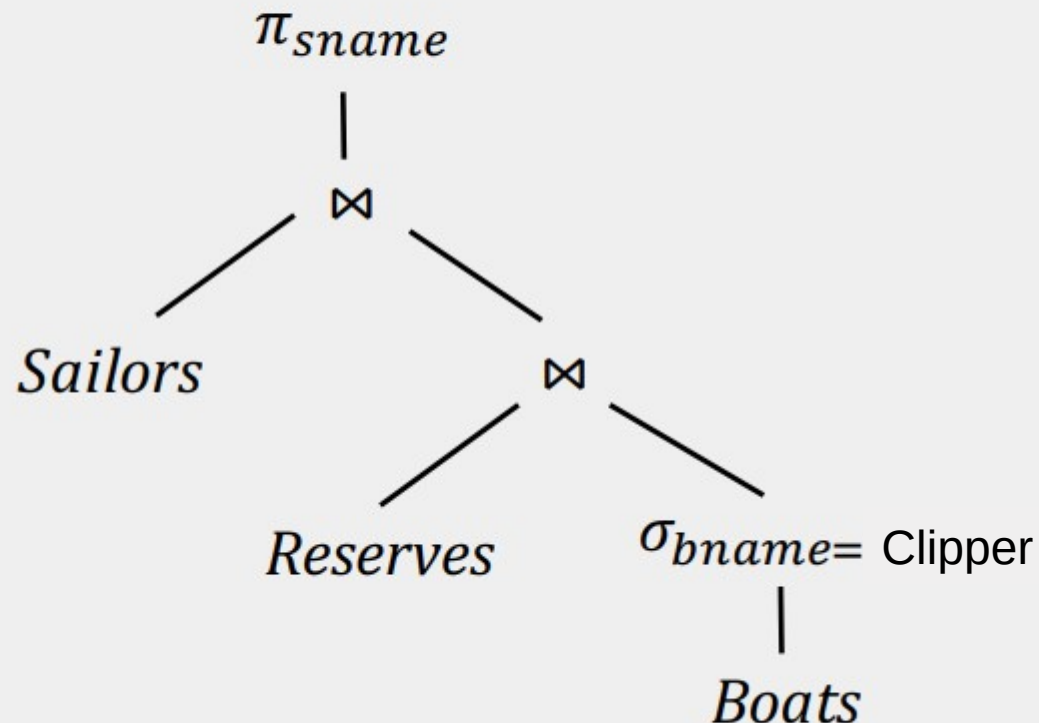
- $|S| = 12$
- $|B| = 4$
- $|R| = 10$
- $f(\text{bname}) = 1/3$
- $f(\text{b.bid}) = 1/4$
- $f(\text{S.sid}) = 1/12$

- $t_S = t_B = t_R = 25$ byte
- $P = 50$ byte
- $P / t_x = 2$
- $P_S = 6$
- $P_B = 2$
- $P_R = 5$

Esempio

- | | |
|---|-------------------------------|
| ▪ $ S = 12$ | ▪ $t_S = t_B = t_R = 25$ byte |
| ▪ $ B = 4$ | ▪ $P = 50$ byte |
| ▪ $ R = 10$ | ▪ $P / t_x = 2$ |
| ▪ $f(\text{bname}) = 1/3$ | ▪ $P_S = 6$ |
| ▪ $f(B.\text{bid} = R.\text{bid}) = 1/4$ | ▪ $P_B = 2$ |
| ▪ $f(S.\text{sid} = R.\text{sid}) = 1/12$ | ▪ $P_R = 5$ |

Q1: *What are the names of the sailors who have reserved boat with name Clipper?*



Esempio: costo (I/O)

■ $ S = 12$	■ $t_S = t_B = t_R = 25$ byte
■ $ B = 4$	■ $P = 50$ byte
■ $ R = 10$	■ $P / t_x = 2$
■ $f(\text{bname}) = 1/3$	■ $P_S = 6$
■ $f(B.\text{bid} = R.\text{bid}) = 1/4$	■ $P_B = 2$
■ $f(S.\text{sid} = R.\text{sid}) = 1/12$	■ $P_R = 5$

- Costo dello scan di Boats:
 - 2 (devo leggere tutte le pagine di Boats e tenere solo le tuple con bname = 'Clipper', ovvero 1)
- Costo del primo JOIN: $P_B + P_B * P_R$
 - $(1 + 3) = 4$ (devo solo fare il lookup – costo 1 – sull'indice di R per trovare le prenotazioni (3) con r.bid corrispondenti al b.bid di Clipper e aprire (alla peggio) 3 pagine per trovare l'r.sid corrispondente)
- Costo del secondo JOIN:
 - $3 + 3$ (faccio 3 lookup dell'indice su s.sid per trovare i nomi dei 3 marinai delle 3 prenotazioni aprendo – alla peggio – 3 pagine)
 - NB: l'output del primo JOIN è già in memoria

Costo totale: $2 + 4 + 6 = 12$

Discussione dell'esempio

- L'ottimizzatore ha adottato una politica di *early filtering* per avere poi dei JOIN più efficienti:
 - È partito da Boats, dove – grazie alla selettività di «Clipper» – è rimasta una sola tupla
 - Ha usato un ordine di Nested Loop Join molto efficiente (Boats → Reserves → Sailors), partendo dal singolo b.bid, facendo un *equality search* su indice in Reserves e poi 3 *equality search* su indice in Sailors
 - Gli indici sugli attributi rilevanti di Reserves e Sailors hanno evitato costosi Table Scan